# CariAgent Product Requirements Document

v0.4 | Product Requirements, Key Technical Decisions, and Iteration Records


- 1. Document Overview
- 2. Background, Users, and Product Goals
  - 2.1 Problem Definition
  - 2.2 Target Users and Core Scenarios
  - 2.3 Goals and Boundaries
- 3. Current Capabilities and Version Scope
- 4. Product Flow and Interaction
  - 4.1 User Main Flow
  - 4.2 System Processing Flow
  - 4.3 Web Pages and Key Actions
  - 4.4 Key Exception Handling
- 5. Detailed Functional Requirements
- 6. AI Behavior, Scoring, and Follow-up Question Rules
  - 6.1 Fact and Evidence Attribution
  - 6.2 Retrieval Rules and Implementation Scope
  - 6.3 Evidence Coverage Score
  - 6.4 Human-in-the-loop
- 7. Architecture, State, and Non-Functional Requirements
  - 7.1 State and Modules
  - 7.2 Quality and Operational Requirements
- 8. Product Evolution and Key Technical Decisions
  - 8.1 Clearly Deferred Technical Directions
- 9. Acceptance, Testing, and Performance Evaluation
  - 9.1 Core Acceptance Scenarios
  - 9.2 Existing Validation Evidence
  - 9.3 Subsequent Measurement Metrics
- 10. Release, Known Issues, and Follow-Up Plans
  - 10.1 Release Baseline
  - 10.2 Current Key Issues
  - 10.3 Next Round Project Initiation Principles
- Appendix A: Materials and Evidence Index
- Appendix B: Document Maintenance Rules

# 1. Document Overview

| Item | Content |
|---|---|
| Product | CariAgent: AI Resume Customization Assistant Based on Real Experience and Target Job Descriptions |
| Document Version | v0.4; this refers to the document version and does not imply a corresponding v0.4 release of the application |
| Last Updated | October 17, 2026 |
| Compilation Method | Summarized and retroactively organized from historical PRDs, development logs, design documents, current code, and test records |
| Current Form | English-only Streamlit web MVP with a retained CLI; the separate main branch maintains the Chinese edition |
| Status Reference | Implementation status based on the current repository; historical decisions preserved for context; benefits not verified are not recorded as proven outcomes |

This document clarifies product goals, interaction and business rules, and acceptance criteria, while consolidating key development trade-offs. The original application baseline was verified at commit `138ed53`. On October 17, 2026, the english-app branch was additionally consolidated into an English-only repository, including prompts, source documentation, guides, diagrams, and PRD exports. The project author reports having created the hosted English app; no verifiable public URL was supplied for this review, so hosted acceptance is not recorded as passed.

# 2. Background, Users, and Product Goals

## 2.1 Problem Definition

Job seekers typically need to repeatedly select and rewrite their experiences tailored to different positions, and a single existing resume does not encompass their complete experience. Source materials often contain colloquial expressions, redundancies, and missing details. Directly generating a resume via a large language model may omit relevant evidence, exaggerate results, or mix facts from different experiences.

CariAgent decomposes resume customization into an inspectable evidence chain:

**Understand the job position → Find evidence → Assess coverage and gaps → User supplementation → Organize the resume.**

## 2.2 Target Users and Core Scenarios

The current priority is students and early-career job seekers who need to customize resumes for specific job descriptions (JDs). Cross-domain job seekers are potential users as well, but tailored adaptation for senior, multifaceted professionals has not yet been validated.

Typical scenario: Users possess multiple internships, projects, and campus experiences and select the most relevant facts for a target position. The system identifies information that can be supplemented; users may choose to answer or skip, ultimately obtaining an editable resume draft.

User needs and issues are primarily based on the author's actual job-seeking experience, self-testing, and limited trial feedback, and do not represent comprehensive large-scale user research.

## 2.3 Goals and Boundaries

| Goal | Product Requirement |
|---|---|
| Select relevant experiences according to the job position | Retrieve evidence based on the target JD, avoiding generic polishing only |
| Maintain factual accuracy | Facts must be attributable to original experiences or user supplements; no fabrication or cross-experience evidence borrowing |
| Support user involvement | Users can understand gaps, answer or skip, and end supplementation voluntarily |
| Deliver usable drafts | Resume content and optimization suggestions are displayed separately; downloads contain only resume content |
| Provide English usage pathway | English interface, prompts, follow-up questions, generation, and export all use a consistent language |

Currently excluded features: automatic job search and application, account and payment systems, cross-session persistent fact repository, production-grade multi-tenant isolation, OCR/visual parsing, premium PDF formatting, job offer prediction, and the already completed LangGraph multi-agent architecture.

## 3. Current Capabilities and Version Scope

| Module | Current Deliverables |
| --- | --- |
| Input | Text editing; extraction of content from TXT, Markdown, DOCX, and text-based PDF; built-in virtual examples |
| JD Understanding | Structuring of job goals, responsibilities, KSAs, baseline conditions, and keywords |
| Evidence Processing | Parsing of sections and individual experience entries; hybrid retrieval; source information; Experience Atoms |
| Matching | Quality check of materials; assessment of requirement coverage; procedural scoring; adaptation explanation |
| User Supplement | Question-by-question answering on web; skip option; view reasons for questions; direct generation |
| Output | Markdown-format resume body; standalone optimization suggestions; Markdown/DOCX downloads |
| Language | English entry point `ui/app_en.py`; `ui/app.py` is also English on this branch. The Chinese edition is maintained on main |
| Development Diagnostics | API call metadata; workflow trace; optional intermediate JSON/Markdown files |

Although image file extensions appear in the upload control, the current code prompts users to paste text instead and does not support image understanding. The web interface does not write generated results into the outputs directory by default; session caching does not equate to long-term data storage.

# 4. Product Flow and Interaction

## 4.1 User Main Flow

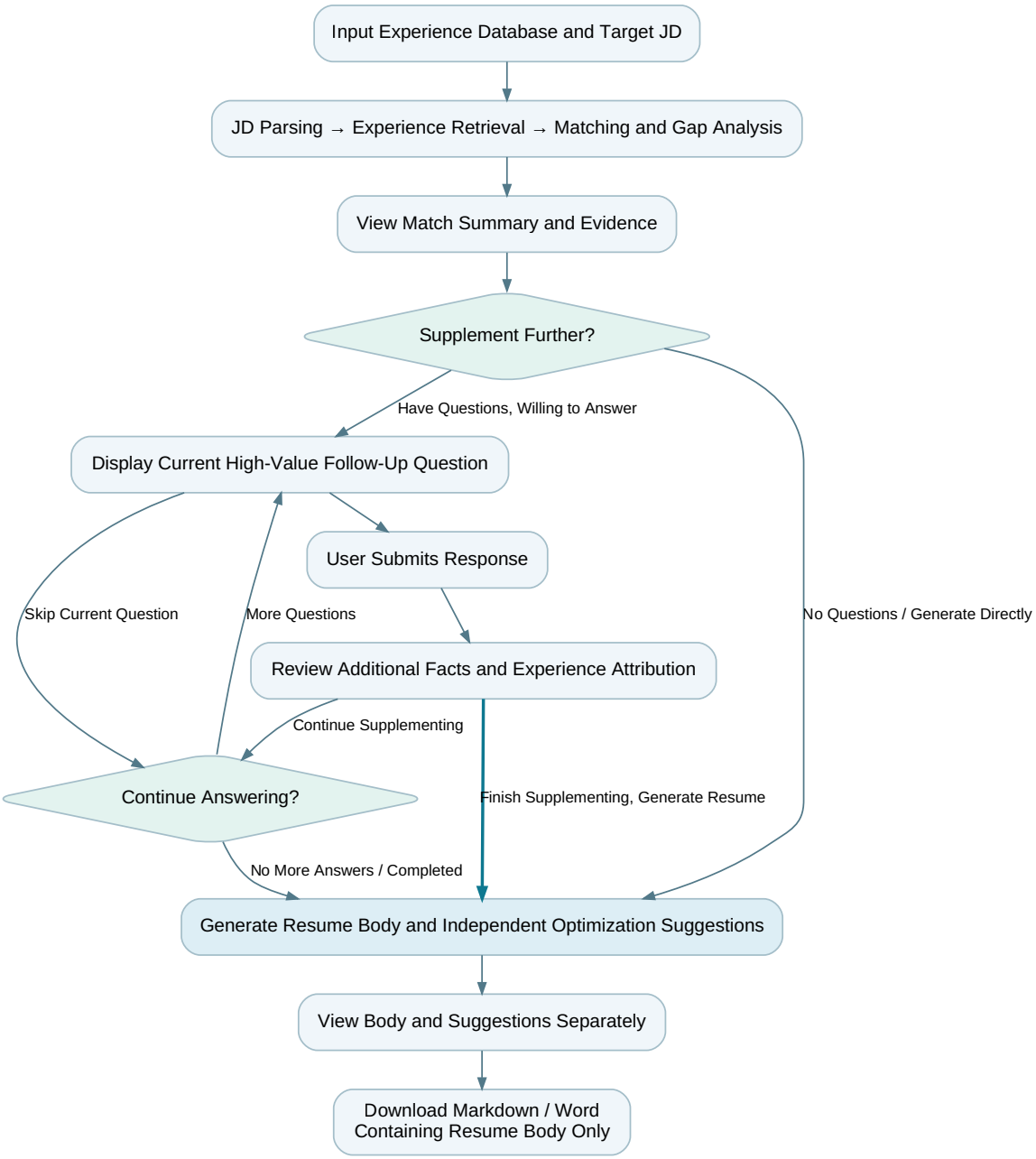



Figure 1: User Main Flow; After Supplementary Fact Verification, Resume Can Be Generated Directly

In the diagram, “Review and Supplement Facts and Experience Attribution” can directly proceed to “Generate Resume” or continue to process the next question; returning a matching summary is not required, and answering all questions is not mandatory.

**Operation Boundary:** Users must submit the current answer before clicking generate. The current generate button does not automatically submit any unsaved textbox content.

## 4.2 System Processing Flow

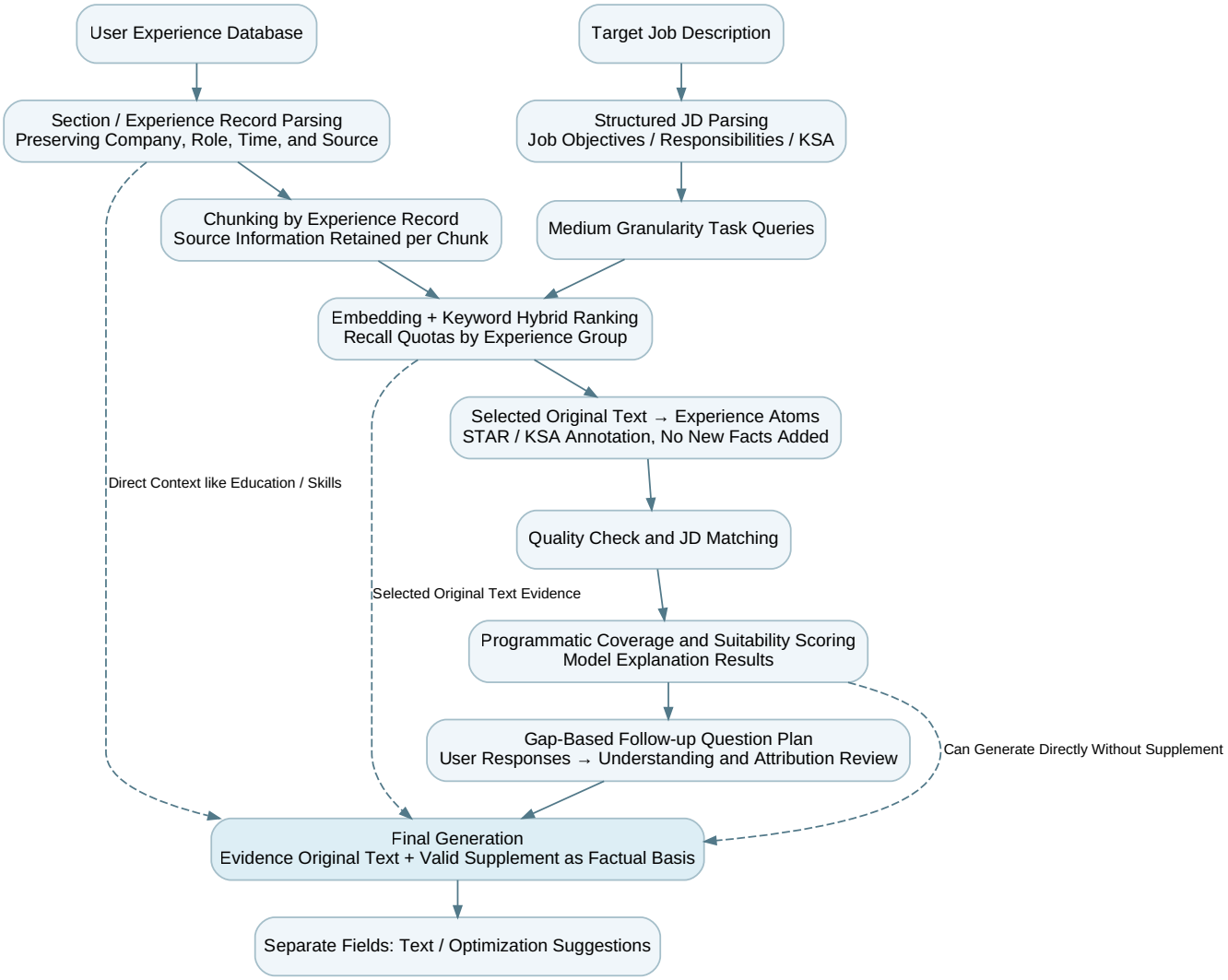



Figure 2: System Evidence Processing and Generation Flow

| Stage | Input → Output | Responsibility Boundary |
|---|---|---|
| JD Parsing | JD → Structured Job Requirements | Do not arbitrarily expand unlisted detailed skills in the JD into independent requirements |
| Experience Parsing | Experience Database → Sections / Experience Records | Classify and preserve facts; do not split tasks within the same company, role, and time period into multiple experiences |
| Evidence Retrieval | Experience Chunks + JD Task Units → Selected Chunks | Retrieve original passages with source retention; do not directly generate resumes |
| Structuring & Review | Selected Evidence → Atoms, Material Quality Check | Tag STAR/KSA elements; do not re-read the entire experience database to bypass retrieval |
| Matching & Scoring | JD + Atoms → Coverage, Gaps, Scores and Explanations | Model classification plus programmatic scoring; explanations must not overextend conclusions |
| Follow-up & Understanding | Gaps + Answers → Verified Supplement | Do not supplement facts on behalf of users; do not auto-write back into original experience database |

| Stage | Input → Output | Responsibility Boundary |
|---|---|---|
| Final Generation | Original Evidence + Valid Supplements + Necessary Tags → Two Output Types | Separate main text and suggestions; internal fields are not exposed |

### 4.3 Web Pages and Key Actions

| Page | User Action | Outcome |
|---|---|---|
| Experience Database | Edit, Import, Continue | Loads experience as input for current process; allows viewing parsed experience categories |
| Target JD | Edit, Import, Analyze | Displays job requirements and keywords, then proceeds to matching |
| Matching Summary | View fit explanations, strengths, gaps, and evidence | User decides whether to supply supplements; low scores serve as recommendations, not stop decisions |
| AI Follow-up | View reasons, submit, skip, return, direct generate | Submitted supplements apply to current round; answering all questions is not mandatory |
| Resume Draft | View main text and separate suggestions, download | Markdown and Word files contain main text only |

### 4.4 Key Exception Handling

| Situation | Expected Behavior and Current Limitations |
|---|---|
| Empty Input | Prompt for additional input before initiating valid analysis |
| Unsupported file or parsing failure | Display the cause; allow fallback to plain text input; do not mock successful reading |
| Missing Key or API call failure | Display error while retaining page usability; no guaranteed automated recovery at any node |
| No candidate follow-up questions | Allow direct resume generation |
| “Don’t know,” “None,” or Skip answers | Accept information boundaries; do not infer nonexistent results |
| Material updates | Do not treat previous input analysis as a new result; current cache keys include input and language, but editing sync and old state cleanup require dedicated validation |
| Refresh, navigation away, or service restart | No promise of cross-session recovery; internal page switching and long-term saving are distinct capabilities |

## 5. Detailed Functional Requirements

P0 indicates core path necessity, P1 indicates auxiliary understanding or development efficiency. Acceptance criteria describe behaviors; verification status is in Chapter 9.

| ID | Priority | Requirement and Key Rules | Acceptance Criteria |
|---|---|---|---|
| IN-01 | P0 | Input complete experience database and one target JD (job description), supporting declared text file formats | Both inputs can be analyzed; empty inputs or unsupported types return explicit feedback |
| JD-01 | P0 | Use KSA (Knowledge, Skills, Abilities) to understand the position; retain baseline education, years independently | Structured output reflects original JD; no arbitrary addition of minor requirements |
| EV-01 | P0 | First parse sections and individual experience paragraphs; then retrieve within experience group | Work/internship and project evidence recalled independently; each block has identifiable source |
| EV-02 | P0 | Only selected evidence is structured; separate facts and annotations | Atom correctly linked to related experience; conflicts resolved based on original text and user responses |
| MT-01 | P0 | Provide coverage judgment, gaps, and algorithmically computed adaptation level | Confirmed match labels produce definitive scores; textual explanation does not re-score |
| HU-01 | P0 | Follow-up questions sorted by high-value gaps; explain question purpose | User can see current question, related requirements/experience, and can skip questions |
| HU-02 | P0 | Review submitted answers; retain fact attribution only for current round | Answers are question-related; do not mix with other experience segments; do not auto-update base fact database |

| ID | Priority | Requirement and Key Rules | Acceptance Criteria |
|---|---|---|---|
| HU-03 | P0 | Users can finish supplements and generate resumes | Direct generation path after review; no need to return to matching summary or answer all questions |
| RES-01 | P0 | Output resume main text and optimization suggestions as two separate fields | Page sections display separately; Markdown and DOCX export only main text |
| RES-02 | P0 | Retain real companies, roles, and known dates; no fabricated quantified achievements | Output traceable to evidence; use conservative expressions for missing info, no fake numbers |
| EN-01 | P0 | English interface covering UI, prompt words, local follow-up questions, error fallback, and output | English example flows contain no Chinese remnants; do not modify schema keys, enums, or fact associations |
| OBS-01 | P1 | Retain debuggable state and call metadata | Enable locating phase, input/output, and call volume; logs do not record keys |

**Status Consistency Supplement:** v0.3 documented a fix separating edit drafts from business inputs, but current code shows no implementation with the same name. This issue remains to be verified and cannot be marked resolved solely on previous documentation.

# 6. AI Behavior, Scoring, and Follow-up Question Rules

## 6.1 Fact and Evidence Attribution

Sources of facts include only original experiences, retrieved source texts, and user supplements. JD analysis, atoms, matching tags, scores, and summaries serve as auxiliary annotations and do not constitute new facts.

- Each piece of evidence retains the association with section, company/project, specific experience, and related questions throughout the process.
- Tasks within the same experience can be combined in expression, but actions or achievements from different experiences must not be merged.
- Information denied or unverifiable by the user is not written as fact; missing Results should not be fabricated.
- The main text must not include chunk IDs, internal scores, retrieval metadata, or debug analysis.
- Model outputs are constrained by schema structure; even with a correct structure, factual accuracy and attribution must be verified.

## 6.2 Retrieval Rules and Implementation Scope

| Item | Current Implementation |
|---|---|
| Query | Job objectives and primary responsibilities formed into medium-granularity task units, supplemented with keywords / KSAs |
| Document Granularity | Original experience chunks; default length 620 characters with 100 characters overlap |
| Ranking | Combined semantic similarity and keyword overlap; default weighting 0.72 / 0.28 |
| Recall Group Quota | Work/internship 12, projects 6, campus/others 3; quotas apply to chunks, not to resume bullets |
| Direct Context | Education, skills, certificates do not compete in the same recall, reserved as background context |
| Embedding Model | `text-embedding-3-small` |
| Persistence | Runtime embedding similarity computation in memory; no deployed persistent vector database |

These parameters represent the current implementation baseline and are not optimal for all user groups. The final output preserves the selected source text and by default does not truncate; priority is given to reducing repetitive intermediate context before considering saving by losing original evidence.

## 6.3 Evidence Coverage Score

The Readiness score measures the degree to which the current materials provide evidence coverage for the JD; it is not a hiring probability or a comprehensive candidate ability score.

`Internal score = Σ (requirement weight × coverage score) / Σ requirement weight`; internal range 0–1, displayed as a percentage.

| Coverage Level | Strong | Medium | Weak | None |
|---|---|---|---|---|
| Coverage Score | 1.00 | 0.65 | 0.35 | 0.00 |

Current importance mapping: high → 1.5; medium → 1.0 (if an ability then 0.8); low → 0.6. Coverage level is calculated conservatively from the model's match level combined with evidence quality.

| Percentage Range | Fit Level | Display Meaning |
|---|---|---|
| ≥85 | high_fit | Strong coverage, suitable for continuing resume organization |
| ≥70 and <85 | good_fit | Good coverage |
| ≥55 and <70 | partial_fit | Partial coverage, supplement cautiously |
| ≥40 and <55 | weak_fit | Limited coverage |
| <40 | low_fit | Insufficient support for core requirements; reconsider recommended |

Program calculation minimizes score drift, but the model still participates in preliminary classification; therefore, reruns do not guarantee identical scores. Early logs using multifactor multiplication and STAR completeness are exploratory records, not part of the current total score formula.

### 6.4 Human-in-the-loop

The current web interface generates candidate questions based on matching gaps and displays them sequentially by priority; users may submit, skip, or generate questions at any time. The understanding of answers preserves usable facts and experience associations; supplementary estimated scores on the page should not be regarded as actual improvements from a full rematch.

Differences between historical mechanisms and the current entry point:

| Item | Current Web Behavior | CLI / Historical Mechanism |
|---|---|---|
| Initial Questions | Generated based on gaps and displayed in order | Separate topic pool and interaction controls |
| High Score Early Stop | The page does not forcibly skip all follow-ups based on a 0.8 threshold | CLI applies readiness threshold judgment |
| Deep Dive on Same Topic | The current web mainly handles pre-existing candidate questions | CLI supports dynamic topic follow-ups, clarifications, and budget control (current constants: total questions limit 10, additional questions per topic 2, clarification budget 1) |
| User Ends Interaction | Can directly click generate | Controlled by specific end commands |

"Following up deeply based on answers" is an important area of ongoing R&D; it cannot be claimed that the current web interface fully adopts CLI's dynamic follow-up mechanism.

## 7. Architecture, State, and Non-Functional Requirements

### 7.1 State and Modules

Currently, the system is a multi-step AI application with shared workflow functions and unified state management. It has not yet been migrated to a LangGraph multi-agent system. The user experience maintains a continuous understanding from a single assistant perspective, though individual model calls do not need to inherit the full conversation context.

| State Layer | Stored Content |
|---|---|
| Raw Facts | Experience database, job descriptions (JD), supplementary answers |
| Evidence Retrieval | Sections, selected chunks, provenance |
| Structured Annotation | Atoms, quality checks, matches, and gaps |
| Decisions and Interaction | Scores, adaptation conclusions, questions, review results, branch records |

| State Layer | Stored Content |
|---|---|
| Presentation and Diagnostics | Main content, recommendations, traces, API payload metadata |

Key modules: `core/simple_mvp.py` manages steps and state; `core/retrieval.py` handles retrieval; `prompts/` manage prompt templates; `ui/app.py` provides common user interface pages; `ui/app_en.py` launches the English mode.

### 7.2 Quality and Operational Requirements

| Dimension | Requirements and Current Limitations |
|---|---|
| Correctness | Does not fabricate facts or mix experiences; relies on examples and manual verification; no large-scale factual accuracy measurement available yet |
| Waiting Experience | Displays progress and errors during presentation stage; logs show single wait times around 3 minutes 20 seconds but this is not a stable performance baseline |
| Stability | Supports call retries and page error handling; no checkpoint across restarts, no full background task recovery, or stable SLA established yet |
| Privacy | API keys injected via runtime environment or Streamlit Secrets; documents and keys are not uploaded to GitHub; source texts are sent to model/embedding services—privacy notices still need improvement |
| Storage | Web default `CARIAGENT_SAVE_OUTPUTS=0`; sessions and cache may retain runtime data; cross-session storage is not guaranteed nor is zero retention ensured |
| Cost | Records call metadata and usage; passes necessary context step-by-step; no experimentally verified cost savings ratio yet |
| Language | This branch publishes only English-facing assets. The separate main branch retains the Chinese edition; source attribution and facts remain unchanged |

## 8. Product Evolution and Key Technical Decisions

The following is arranged according to verifiable stages from the materials; stages without explicit dates do not have fabricated dates or release versions. Historical “resolved” entries reflect conclusions at the time and do not replace current code or test verification.

| Stage | Issue / Observation | Key Decisions and Trade-offs | Evidence and Outcomes |
|---|---|---|---|
| 2026-05-08: Scenario Formation | The author repeatedly reviewed job positions, filtered all experiences, and customized the resume during application | Narrowed down from a fully automatic job application concept to evidence matching and generation between experience database + job description (JD); retained user judgment | Development logs record the original job application path and scope considerations; automatic submission not implemented |
| Early stage: Follow-up questions and explainable scoring | Generating all content made it hard to detect omissions; relying solely on “AI thinks it’s sufficient” cannot explain when to stop | Introduced STAR/KSA frameworks, explicit matching, importance weights, gaps, and stopping conditions | Logs screenshots retained multiple scoring and follow-up query schemes; some formulas were later replaced |
| Third and fourth self-tests: Depth of follow-up queries | Multiple gaps existed but only one question was asked; when answers mentioned another experience segment, natural continuation was desired; wait times were long | Explored prioritization, clarification and deepening within same topic; distinguished “unclear answers” from “valuable to continue” | 2026-05-16 logs recorded experience improvements, but it was still slow; no confirmed overall improvement ratio |
| RAG (Retrieval-Augmented Generation) introduction: from full reading to retrieval | Increasing context size, irrelevant experiences diluted attention, increased cost and latency pressure | Made evidence selection explicit: chunking, embeddings, semantic + keyword hybrid; prepared for on-demand reading by JD | Development and RAG logs; reduced subsequent full content passing, though initial section parsing still reads entire experience database |
| Query correction: Fine KSA to task units | Abstract fine queries resulted in keyword-like patterns and factual mismatches causing false recalls | Switched to medium-grain queries based on job objectives and responsibilities; KSA continued assisting understanding but no longer directly equal to fine retrieval units | RAG logs document corrections; current code retains task units; no strict before/after recall rate experiments |

| Stage | Issue / Observation | Key Decisions and Trade-offs | Evidence and Outcomes |
|---|---|---|---|
| Recall structure: Section-aware retrieval | Project information density overshadowed work/internship evidence | Independent retrieval and quotas per experience group; education and skills directly included in context | RAG logs and current code; professional experience area retained, quotas need evaluation for different user groups |
| Fact boundary: Provenance | At times, content from one organizational experience was assigned to another experience segment | Propagated provenance from section, experience record, chunk to atom / match / answer; avoided abstracts being treated as new facts | RAG logs contain misassigned case examples; provenance fields serve as constraints, not proof of zero errors |
| Step responsibility narrowing and context control | Step 4 implicitly inherited all materials, possibly bypassing retrieval, causing token overload | Step 4 only structures selected original text; separate step calls with only necessary context and unified state maintenance to ensure continuity | Workflow documentation and code; final generation still prioritizes preserving selected original text rather than default truncation to save tokens |
| Stable output and scoring | Model architecture instability and scoring drift possible | Used JSON schema; LLM conducts classification and explanation, program computes total coverage scores and grades | v0.3 and code; classification uncertainty remains, comprehensive stability metrics pending retesting |
| Workflow and web interface | Single script and CLI were unfriendly to non-technical users, state tracking was difficult | `WorkflowState` plus step functions plus Streamlit five-step interactive interface with session caching and diagnostics | 2026-05-28 Git records and v0.3; editing synchronizations in earlier documents differ from current implementation |
| 2026-10-17: Output boundaries and English version | Recommendations might be embedded in download files; UI translation alone still leaked Chinese | One structured call returns two fields; shared core, independent English entry point, full pipeline language coverage | Commit `138ed53`; initial real-world tests found residual prompt conflicts, corrected and retested; English language sample pipeline passed |

### 8.1 Clearly Deferred Technical Directions

| Direction | Reason for Deferral / Conditions for Re-evaluation |
|---|---|
| Persistent vector database | Current single experience database and in-memory retrieval suffice; multi-user, multi-file, duplicate retrieval, and long-term cross-session storage needs to be established first |
| Personal experience knowledge graph | Early conceptual stage; current core path implemented using original text evidence, provenance, and structured annotations |
| LangGraph / Multi-agent systems | First clarify states, steps, input/output; considered when checkpointing, complex branching, node retry, or parallel collaboration is required |
| Long-term fact database | Values include cross-JD reuse and incremental updates; current lacks account management, write-back confirmation, and persistence management |
| Multi-modal understanding | Current text extraction does not equate to OCR, visual, or multi-modal retrieval; requires independent validation |

## 9. Acceptance, Testing, and Performance Evaluation

### 9.1 Core Acceptance Scenarios

| Scenario | Acceptance Criteria |
|---|---|
| Standard Complete Path | Experience and JD can complete parsing, retrieval, matching, follow-up questioning, and generation |
| Missing Deliverables | Accept missing content; do not fabricate numbers or treat exercise tasks as real business results |
| Experience Attribution | Actions, outcomes, and follow-up supplements from two experiences must not be cross-used |
| Skipping and Early Generation | Not required to answer all questions; submitted answers are used, unsubmitted content is not pretended as reviewed |

| Scenario | Acceptance Criteria |
| --- | --- |
| Main Text / Suggestions | Pages separated; Markdown/DOCX files contain only the main text |
| English Consistency | English examples are coherent throughout pages, questions, review, main text, and suggestions with no Chinese residuals |
| Editing and Navigation | Updates on input, back, and continue do not lose the latest text; no outdated results used; special confirmation required |
| Error Recovery | API or file parsing failures provide understandable feedback; partial failures are not described as complete |

## 9.2 Existing Validation Evidence

| Record | Confirmed Results | What It Does Not Represent |
| --- | --- | --- |
| Recorded Baseline Offline Regression | 6 tests passed: output separation, prompt boundary, page/download, as well as English prompts and navigation, etc. | Does not cover all interactions and exceptions |
| Chinese Real Samples | Built-in experience + Data Analyst internship JD parsed and generated correctly; retesting passed after fixing suggestions mixed into main text | No real-time human follow-up acceptance completed; partial time information loss discovered previously |
| English Real Samples | 8 model calls plus real retrievals; 3 candidate follow-up questions; simulated answer review, main text, and suggestion generation passed | Not 3 separate users or 3 rounds of real human testing, nor large-scale efficacy conclusions |
| English Page and Export | Actual content filled five pages; English copy, Markdown, and Word outputs verified; internal service health checks passed | Not equivalent to online deployment or external access acceptance |
| Previous Trial Use | Author reported about 15 participants experienced and about 3 continued use during interview material | Not a unified time window with defined 20% retention and instrumentation |

The English-only repository cleanup adds an eight-test offline suite covering repository text, PDF and Word language, English UI navigation, model prompt contracts, and resume/advice separation. Real API smoke validation is recorded separately from offline checks.

## 9.3 Subsequent Measurement Metrics

| Metric | Definition | Current Status |
| --- | --- | --- |
| Process Completion Rate | Number of effective sessions generating main text / number of sessions with submitted valid input during test period | Session and event definitions to be established; no baseline yet |
| Waiting Time | Time recorded separately for initial match completion, answer submission completion, and final generation | Single observations only; no reliable p50/p95 yet |
| Factual Errors | Count of fabricated facts, cross-experience misattributions, and key fact omissions in manually checked samples | No systematic annotated dataset yet |
| Follow-up Value | Number of answers with effective new facts / number of submitted answers, combined with user evaluation | Simulated calls cannot substitute real user results |
| Adoption | Whether users export, modify and reuse, or reuse for another JD | Awaiting trial feedback and compliance measurement |

The v0.3 proposed stability acceptance goals remain candidates but have **not been accepted based on this round of testing**: running the same input 3 times with score gaps no more than 5 percentage points, no grade drift exceeding one adjacent grade, and at least 60% overlap in strengths and gaps. Formal measurement requires fixed models, prompts, configurations, and explicit overlap matching criteria.

# 10. Release, Known Issues, and Follow-Up Plans

## 10.1 Release Baseline

- The Chinese edition is maintained on `main`. This `english-app` branch contains English-only application files and project documentation. Commit `138ed53` introduced the original English runtime and separated resume/advice output; subsequent cleanup removes Chinese fallback text and documents without changing the scoring or retrieval algorithm.
- The English application uses `ui/app_en.py`, deployed separately from the Chinese entry point; keys are configured independently in the corresponding hosted applications and are not committed to the repository.
- The tests listed in this document target code in the development environment. Production release requires verification of branch, entry point, keys, actual calls, file downloads, and real browser experience.
- In case of regression, revert to validated code commits and ensure related prompt sync is also rolled back; do not only revert page files.

## 10.2 Current Key Issues

| Issue | Handling Direction |
|---|---|
| Known dates become “Time unspecified” during generation | Inspect record parsing, chunk/atom metadata, and final context passing; fix and test independently |
| v0.3 editing sync records are inconsistent with current implementation | Verify latest text submission, navigation, and old analysis cleanup using real browsers |
| Webpage and CLI have different follow-up questioning capabilities | Describe product behavior according to actual webpage; migrate dynamic follow-up questioning as a separate project before integration |
| No long-term data management, breakpoint recovery, or mature monitoring currently | Prioritize based on real-world usage frequency and failure cases; avoid expanding framework names unnecessarily |

## 10.3 Next Round Project Initiation Principles

First gather feedback that impacts full product use, factual trust, and delivery quality, then select a round of changes with clear goals. Options such as switching cloud platforms, introducing Agent frameworks, or expanding promotion are candidates but are not committed features. Each round should add requirement IDs, acceptance scenarios, code commits, and actual results to be continuously maintained in this document.

# Appendix A: Materials and Evidence Index

| Source | Usage in This Document |
|---|---|
| `cari_agent_prd_v_0_1.md` | Confirmation that the old version is deprecated; missing main text, thus not used to restore specific functionalities of the first version |
| `cari_agent_prd_v_0_2.md` | Local MVP, RAG, evidence boundaries, scope, and pending tasks at the time |
| `CariAgent_PRD_Portfolio_v0_3.pdf` | Web phase, scoring, interaction records, stability goals, and portfolio explanation |
| Development diary (original DOCX, including screenshots) | Ideas as of 2026-05-08, self-testing with follow-up queries, scoring exploration, RAG motivation, and directions temporarily paused |
| `cari_agent_rag_development_log.md` | Key corrections regarding query granularity, sections, provenance, Step 4, etc. |
| `workflow_design_notes.md` | Unified status, stepwise responsibilities, original text priority, CLI thresholds, and context trade-offs |
| Repository source, Git history, and recorded development test artifacts | Core implementation actions, English and Chinese modifications, test results, and yet uncovered scope |
| Interview transcripts provided by the author and dialogue from this round | Trial feedback, English version goals, and current deployment status narration; separated from runtime evidence |

The organization references the publicly available Microsoft PowerToys specification review process and SCOOBE feature specification: structured by problem, goal, priority, behavior, measurement, and prototype, linked to R&D validation. No fictional target values from any public templates have been adopted as project outcomes.

- PowerToys Specification Review Process
- PowerToys SCOOBE Feature Specification

Additional suggested references were unavailable during the review and are not treated as verified evidence.

## Appendix B: Document Maintenance Rules

Requirement changes update Chapter 5; major solution trade-offs update Chapter 8; testing and release update Chapters 9–10. Maintain the distinctions among “Observation, Decision, Implementation, Verification,” without adding nonexistent R&D dates, team reviews, or performance metrics.

The flowchart source files are `assets/user-flow.dot` and `assets/system-flow.dot`, maintained alongside their corresponding SVG files. Markdown serves as the primary maintenance file; web pages and PDFs are generated from the same source file to avoid discrepancies caused by multiple independently edited versions.